

Topology Operating Canvas Guide

Updated Enterprise Architecture

Deep research update for building futuristic, extensible, flexible, visually stunning topology rendering for large enterprise networks

Executive decision

Do not build a single LLDP/CDP visualizer or a single React Flow graph. Build a topology intelligence platform with multiple renderers: React Flow for scoped operator workflows, Sigma.js/cosmos.gl for large-scale graph exploration, MapLibre/deck.gl for WAN and geo views, and ELK.js/MSAGL-style layout research for deterministic and semantic zoom layouts.

Prepared from the original Correlix Topology Operating Canvas Guide and updated with official public documentation, RFCs, and open-source tooling research.
Prepared: June 17, 2026

Contents

1. What changed from the original guide
2. Vendor patterns worth copying - not vendor internals
3. Product north star
4. Recommended open-source stack v2
5. Target architecture v2
6. Multi-renderer strategy
7. Topology fact model and graph contract
8. Discovery and source-of-truth strategy
9. Map workflows and overlays
10. Semantic zoom, edge bundling, and anti-hairball rules
11. Visual design system and interaction model
12. Layout strategy
13. Performance and scale policy
14. Time, diff, golden path, and digital twin
15. Backend storage and API boundaries
16. Frontend implementation structure
17. Claude Code skill pack v2
18. Implementation roadmap
19. Claude Code prompt
20. Acceptance criteria
21. Public source references

1. What changed from the original guide

Verdict

The original guide is directionally strong. Keep the idea of a topology operating canvas, not an LLDP visualizer. The key upgrade is to make the architecture explicitly renderer-agnostic and enterprise-scale from day one.

Original direction	Keep / change	Updated recommendation
React Flow + ELK.js as first build	Keep	Use React Flow/xyflow + ELK.js for scoped, premium operator canvas views.
Cytoscape.js or Graphology for graph intelligence	Keep and sharpen	Use Graphology as the client graph object; use Cytoscape.js where headless analysis, algorithms, and browser visualization add value.
Sigma.js later for large graph mode	Make explicit now	Create a planned secondary renderer for 1,000+ node enterprise overview mode using Sigma.js or cosmos.gl.
Map modes and overlays	Keep	Rename them as operator workflows: Explore, Investigate, Path Trace, Change Review, Capacity, Dependency, Executive/Geo.
PDF focuses on LLDP/CDP and NOC overlays	Expand	Add BGP-LS, IS-IS/OSPF LSDB, NetBox/Nautobot, cloud APIs, Kubernetes, flow logs, time-diff, and golden path as first-class inputs.
Claude Code skill files	Expand	Add renderer-strategy, semantic-zoom, fact-model, scale-policy, edge-bundling, time-diff-golden-path, and performance-budget files.

Non-negotiable architecture rule

The frontend must never consume raw LLDP/CDP/BGP-LS/SNMP rows. It should consume a resolved topology graph contract with evidence, confidence, source facts, timestamps, ownership, health, and layout intent.

2. Vendor patterns worth copying - not vendor internals

Public documentation does not reveal the private rendering engines used by every vendor. The safe engineering lesson is to copy proven product patterns and build them with publicly available tools.

Vendor / source	Publicly visible behavior	Correlix design lesson
Datadog Network Device Topology Map	Interactive physical network visualization; Datadog docs describe Cloudcraft diagrams and LLDP/CDP-based discovery/resolution for device topology. [1]	Physical topology must combine LLDP/CDP, interface inventory, health, unresolved device handling, and drill-down into upstream/downstream impact.
Datadog Cloud Network Map	Directional network map for partitions, dependencies, and bottlenecks. [2]	Separate physical adjacency from traffic/dependency maps. Flow edges are directional and should show volume, latency, retransmits, and dependency strength.
Dynatrace Smartscape	In-context topology view helps operators understand vertical and horizontal entity relationships; concepts include nodes, edges, groups, filters, and problems. [3][4]	Build an entity graph, not just devices: service -> workload -> host -> interface -> device -> path -> cloud.
NetBrain Dynamic Maps / Digital Twin	Dynamic maps are created from live network data and used for path, data views, baseline, historical comparison, golden state, and automation workflows. [5][6]	Make maps operational: A-to-B path, golden path, historical path, config drift, live evidence, and change-impact workflow.

Product conclusion

Correlix should be closer to a map-driven investigation workspace than a diagram repository. The map must answer: what is affected, what changed, what path is active, what evidence proves it, and who owns the fix.

3. Product north star

North star

Discover -> resolve identity -> model graph -> visualize -> search -> expand -> path trace -> overlay health -> compare change -> explain RCA -> assign owner -> recommend next action.

- Build an operating canvas, not a passive topology screen.
- Default view must be calm. Only selected, unhealthy, changed, or RCA-relevant objects should become visually intense.
- Every link must explain why it exists: LLDP, CDP, BGP-LS, LSDB, NetBox, cloud API, flow, trace, metric, syslog, config, or manual source.
- Every edge and node must carry confidence, evidence, source facts, first_seen, last_seen, and ownership metadata.
- Never show the whole enterprise graph by default. Start from scope, site, path, incident, search result, app, or change window.

4. Recommended open-source stack v2

Layer	Recommended public tool	Decision
Premium operator canvas	React Flow / xyflow	Primary renderer for scoped, high-fidelity NOC workflows: custom nodes, handles, minimap, controls, side drawers, interactions, and React component polish. React Flow performance guidance warns to manage large node counts and complex nodes carefully. [9][10]
Deterministic layout	ELK.js / Eclipse Layout Kernel	Default auto-layout for hierarchical network diagrams. ELK layered supports directional layered layouts, edge routing, and ports; ELK.js makes ELK available in JavaScript. [11][12]
Graph model and algorithms	Graphology + Cytoscape.js	Graphology provides a robust JS/TS graph object; Cytoscape.js is an open-source graph theory library for analysis and visualization, including headless use. [13][14]
Large enterprise overview	Sigma.js or cosmos.gl	Use WebGL renderer when graph exploration exceeds React Flow scope. Sigma.js is WebGL-based and built with Graphology; cosmos.gl runs layout and rendering on the GPU for very large graphs. [15][16]
Geo/WAN/cloud-region map	MapLibre GL JS + deck.gl	Use for sites, branches, regions, WAN circuits, cloud regions, and route/latency overlays on a map. MapLibre uses WebGL vector tiles; deck.gl is GPU-powered for large-scale visual analysis. [17][18]
Future R&D layout	MSAGLJS / semantic zoom research	Track MSAGLJS and tile-pyramid/sleeve-routing approaches for browser-side semantic zoom and large graph routing. [19][20]
Design system	Tailwind + Radix/shadcn + custom SVG icons	Use public UI primitives, strict tokens, and vendor-neutral custom icons for premium enterprise appearance.

Tooling recommendation

Keep React Flow + ELK.js as Phase 1. Add Graphology immediately for a clean graph object. Add Sigma.js/cosmos.gl as a planned renderer adapter, not as a last-minute rewrite. Add MapLibre/deck.gl when WAN/geo/cloud topology appears.

5. Target architecture v2

Topology Sources

LLDP / CDP / SNMP / BGP-LS / IS-IS LSDB / OSPF LSDB / NetFlow-IPFIX-sFlow / Syslog / Metrics / NetBox-Nautobot / Cloud APIs / K8s

|

v

Topology Fact Store

raw facts -> normalized facts -> source evidence -> time validity

|

v

Identity Resolution

inventory ID -> chassis ID -> mgmt IP -> sysName -> hostname -> aliases -> serial -> fuzzy match -> ownership

|

v

Resolved Correlix Graph

nodes + edges + groups + overlays + evidence + confidence + health + ownership + timestamps

|

v

View Builder

explore / investigate / path_trace / change_review / capacity / dependency / executive_geo

|

v

Layout Service

ELK layered / orthogonal / grouped / path-first / radial / geo / saved operator layout

|

v

Renderer Adapter

React Flow canvas / Sigma-cosmos overview / MapLibre-deck geo / export SVG-PNG-PDF

|

v

Operator Workflow

search + spotlight + expand + RCA + evidence + owner + next action + runbook automation

6. Multi-renderer strategy

This is the biggest architectural upgrade. Correlix should not be locked into one rendering library. The domain graph, layout intent, and renderer adapters must be separate.

Renderer	Primary use case	Do not use it for	Notes
React Flow / xyflow	Incident views, site/fabric views, A-to-B path views, edge evidence, side drawers, custom device cards.	Huge unfiltered enterprise graph with thousands of detailed React nodes.	Best for premium NOC UX and rich interactions.
ELK.js	Layout computation for layered, orthogonal, port-aware diagrams.	Rendering UI components directly.	Run on demand; cache layout results.
Graphology	Client-side graph object, traversals, filtering, centrality helpers, Sigma integration.	UI rendering by itself.	Keep this independent from React Flow node state.
Cytoscape.js	Graph analysis and optional headless algorithms/server-side utilities.	Replacing premium React Flow operator canvas.	Useful when graph algorithms and mature layout/analysis ecosystem are needed.
Sigma.js	Large graph overview with WebGL, thousands of nodes/edges, search/filter/cluster exploration.	Highly customized card-like nodes and complex side-panel interactions.	Use as enterprise overview renderer.
cosmos.gl	Very large GPU force graph exploration and cluster discovery.	Deterministic network engineering diagrams.	Candidate for exploratory relationship graph mode.
MapLibre + deck.gl	Geo/WAN/cloud region views, circuits, latency arcs, site health, branch heatmaps.	Rack/fabric diagrams.	Must share the same topology graph contract.

Renderer adapter principle

The UI receives a `TopologyView`. Each renderer has an adapter: `topologyToReactFlow()`, `topologyToSigma()`, `topologyToDeckLayers()`. The graph contract remains stable even if the renderer changes.

7. Topology fact model and graph contract

Why this matters

The moat is not the drawing library. The moat is a resolved, time-aware, evidence-backed topology graph that can be rendered many ways.

7.1 Raw topology fact

```
type TopologyFact = {
  fact_id: string
  tenant_id: string
  source: 'lldp' | 'cdp' | 'snmp' | 'bgp_ls' | 'isis_lsdb' | 'ospf_lsdb' | 'netbox' | 'cloud_api' | 'k8s_api' | 'flow' | 'trace' | 'syslog' | 'metric' | 'manual'
  subject: string
  predicate: string
  object: string
  source_device?: string
  source_interface?: string
  target_device?: string
  target_interface?: string
  observed_at: string
  expires_at?: string
  confidence: number
  raw_ref: string
  attributes: Record<string, unknown>
}
```

7.2 Resolved topology view contract

```
type TopologyView = {
  view_id: string
  topology_id: string
  mode: 'explore' | 'investigate' | 'path_trace' | 'change_review' | 'capacity' | 'dependency' | 'executive_geo'
  scope: { tenant_id: string; site?: string; app?: string; path_id?: string; incident_id?: string }
  generated_at: string
  time_range: { from: string; to: string }
  layout_intent: 'spine_leaf' | 'campus' | 'wan_geo' | 'path_first' | 'incident' | 'dependency' | 'cloud_grouped'
  nodes: TopologyNode[]
  edges: TopologyEdge[]
  groups: TopologyGroup[]
  overlays: TopologyOverlay[]
  evidence: EvidenceRef[]
  renderer_hints: { preferred: 'react_flow' | 'sigma' | 'deck_geo'; max_detail_level: number }
}
```

7.3 Required fields on every node and edge

Object	Required fields
Node	id, label, kind, role, vendor, model, site, zone, ownership, health, confidence, first_seen, last_seen, evidence[], metrics{}, tags{}, coordinates or group membership.
Edge	id, source, target, relationship, protocol/source, source_port, target_port, status, confidence, evidence[], first_seen, last_seen, utilization, errors, change_state, bundle_id, direction.
Group	id, label, group_type, children, health_rollup, collapsed_state, layout_policy, ownership, evidence.
Evidence	source, confidence, observed_at, raw_ref, summary, used_by_rca, ignored_reason, missing_evidence_if_any.

8. Discovery and source-of-truth strategy

Layer	Sources	What it contributes	Confidence guidance
Physical / L2	LLDP, CDP, SNMP IF-MIB, bridge/FDB, LAG data, NetBox/Nautobot.	Device-to-device links, ports, LAGs, VLAN hints, unresolved neighbors.	Bidirectional LLDP/CDP high confidence; one-way discovery probable; unresolved remote chassis visible but muted.
Routed topology	BGP-LS, IS-IS LSDB, OSPF LSDB, routing table snapshots.	Routed adjacencies, TE topology, prefixes, link metrics, SR/MPLS/SRv6 context.	BGP-LS should be first-class for SP/DC/WAN path use cases. RFC 9552 describes BGP-LS distributing link-state/TE topology to consumers. [7]
Live telemetry	gNMI/OpenConfig, streaming telemetry, SNMP counters, vendor APIs.	Interface status, utilization, errors, queue drops, optical power, routing state, config state.	gNMI is designed for configuration retrieval and telemetry stream control/generation over gRPC. [8]
Observed dependency	NetFlow, IPFIX, sFlow, cloud flow logs, eBPF/agent, synthetic probes.	Directional dependencies, app-to-network path, top talkers, volume, latency, retransmits.	Flow-only dependencies should be dashed and lower confidence unless reinforced by inventory/path evidence.
Cloud and virtual	AWS, Azure, GCP, Alibaba APIs, Kubernetes API, CNI, load balancer APIs.	Region/VPC/VNet/subnet/workload/service/load balancer/firewall object containment.	Cloud API object relationships are authoritative for cloud containment, not physical path.
Intent and ownership	NetBox/Nautobot, CMDB, tags, Git, Terraform, device config, manual operator annotations.	Expected topology, owner, criticality, golden path, maintenance windows, business service mapping.	Intent is not live truth. Compare against live facts and show drift.

Do not hide unresolved nodes

Unresolved LLDP/CDP/BGP-LS/cloud neighbors are valuable. Render them as muted nodes with explicit confidence and evidence. They often reveal inventory gaps, shadow devices, or discovery blind spots.

9. Map workflows and overlays

Workflow	Purpose	Default layout	Primary users
Explore	Search, inspect a site, expand 1/2/3-hop neighbors, understand physical or routed relationships.	Layered, grouped, or radial around selected seed.	NOC, network engineer.
Investigate	RCA incident map with affected path, blast radius, evidence used/ignored/missing, owner decision.	Path-first with dimmed unrelated graph.	NOC lead, on-call, escalation engineer.
Path Trace	A-to-B hop view: devices, ingress/egress interfaces, routing choice, NAT/ACL/QoS/security zones.	Left-to-right path-first.	Network engineer, app owner, ISP escalation.
Change Review	Before/after topology, live vs intended, golden path, config drift, maintenance impact.	Diff-aware grouped layout.	Change manager, SRE, network architect.
Capacity	Hot links, ECMP imbalance, errors, drops, CRC, saturation, optical power.	Physical or routed layout with utilization overlay.	NOC, capacity planner.
Dependency	App/service/workload/cloud dependencies and blast radius.	Filtered dependency graph, force only after filtering.	App SRE, platform team.
Executive / Geo	Sites, regions, cloud edges, branch health, provider/circuit health.	MapLibre/deck.gl geo view.	Leaders, NOC managers, customer demos.

Overlay	What it shows	Visual rule
Health	Node/link status, alert count, severity, availability.	Health ring and small badges; never fill entire node red/orange.
Utilization	Traffic, interface utilization, ECMP balance.	Edge width and hover values; cap width to avoid visual chaos.
Errors	CRC, drops, discards, flaps, optics.	Small warning markers on affected link segments.
Routing changes	BGP/OSPF/IS-IS/BGP-LS deltas and path divergence.	Route-change badge and side drawer timeline.
Config drift	Live vs intended/golden config.	Drift badge on node/link; click opens config diff summary.
Flow	Observed communication/dependency.	Directional arrows and volume/latency/retransmit summary.
RCA evidence	Used, ignored, missing evidence.	Evidence timeline plus confidence explanation.
Golden/historical path delta	Deviation from intended path or prior time.	Added/removed/changed highlights; match/diverge summary.

10. Semantic zoom, edge bundling, and anti-hairball rules

Anti-hairball rule

A beautiful topology product is mostly a filtering, grouping, layout, and interaction system. If you draw every node and edge at once, you have already lost.

Zoom level	Visible objects	Hidden or aggregated objects
Global	Regions, cloud providers, major sites, WAN providers, critical application domains.	Individual devices/interfaces hidden behind health rollups.
Site	Campus/DC/branch zones, core/distribution/access, WAN edge, firewall zones.	Access endpoints collapsed; redundant access links bundled.
Fabric	Spine/leaf, border leaf, firewalls, routers, load balancers, key clusters.	Server leaves/access ports collapsed unless selected.
Device	Device cards, role, owner, health, critical metrics, first-degree neighbors.	Detailed ports hidden until edge/node selected.
Interface	Physical/logical interfaces, LAGs, VLANs, speed, utilization, errors, evidence.	Remote low-relevance branches dimmed.
Evidence	Raw fact sources, confidence, timestamps, used/ignored/missing evidence.	No visual clutter; details live in drawer and hover cards.

Problem	Rule
Parallel links	Bundle at zoomed-out levels; display as 2x100G, 4x25G, LAG-10. Expand on click.
Access layer noise	Collapse by building, closet, switch stack, VLAN, or access block.
HA firewalls/load balancers	Render as cluster/group node by default; expand members on demand.
WAN circuits	Show provider/circuit badge at site level; show physical/logical edges only in circuit detail.
Labels	Show labels by importance: selected, unhealthy, criticality, search result, path, owner, changed. Hide the rest.
Animations	Use minimal motion for state transitions only. Never animate everything.
Search result	Spotlight matched object and first-degree neighbors; dim unrelated graph.

11. Visual design system and interaction model

The visual system should feel calm, premium, and operational. It should not look like a colorful academic graph. Use intensity only to direct attention.

Token group	Guideline
Canvas	Soft grid, light/dark support, muted background, subtle depth. Avoid heavy engineering-paper grid.
Nodes	Compact cards around 188px wide; 14px radius; role accent; vendor-neutral icon; hostname; health badge; small metrics row.
Health	OK green, warning amber, critical red, unknown slate. Use rings/badges, not full node fill.
Edges	Muted by default. Selected/RCA path stronger. Width reflects utilization. Dashed/dotted reflects lower confidence or inferred relationship.
Typography	Inter/Aptos-like sans-serif. Hostname 13px/600; metadata 11px; badge 10px uppercase.
Density modes	Executive, Operator/NOC, Engineer/debug, RCA/incident. Each density mode controls labels, metrics, and overlays.
Interaction	Hover highlights first-degree links. Click opens drawer. Incident mode dims unrelated graph. Edge click opens evidence/confidence view.
Accessibility	Color is never the only signal. Use icon, label, pattern, and severity text.

Node anatomy

[role icon] hostname [health]

vendor / model

site / rack / role / owner

CPU 42% | MEM 68% | Links 12 | Alerts 2

Edge hover

leaf1 Eth1/1 -> spine1 Eth1/11

LLDP confirmed | confidence 98%

Utilization 63% | Errors 0.02% | Last seen 22s ago

Evidence: bidirectional LLDP + SNMP ifOperStatus + NetBox expected link

12. Layout strategy

Topology type	Layout strategy	Notes
Spine-leaf / Clos	ELK layered top-to-bottom or left-to-right, port-aware, grouped by pod/rack.	No force layout. Keep deterministic positions and readable parallel bundles.
Campus	Core -> distribution -> access hierarchy with building/closet grouping.	Support collapsed closets and stacks.
WAN	Geo layout using MapLibre/deck.gl plus logical overlay for provider/circuit.	Avoid drawing physical distance as topology truth unless coordinates are known.
A-to-B path	Path-first left-to-right; non-path context dimmed.	Hop cards include ingress/egress interfaces and decision notes.
Incident	Root cause -> affected path -> blast radius -> owner/next action.	Evidence timeline sits in side panel; map remains focused.
Dependency	Filtered graph; use force only after scoping to app/service or blast radius.	Never run a random full-enterprise force graph as default.
Cloud	Provider -> account/subscription -> region -> VPC/VNet -> subnet -> workload/service.	Use compound/grouped layout.

Saved layout rule

Layout may be generated by ELK or geo coordinates, but operators should be able to pin/save positions per site/view. Saved layouts must not corrupt the domain graph.

13. Performance and scale policy

Scale band	Renderer / behavior	Performance policy
0-250 rich nodes	React Flow rich cards and custom SVG icons.	Full hover/click interactions, minimap, side drawer, detailed edges.
250-1,000 nodes	React Flow with grouping/collapse and simpler edges.	Collapse clusters, memoize custom nodes, hide expensive labels, avoid global state reads from node components.
1,000-10,000 nodes	Sigma.js or cosmos.gl overview; React Flow only for selected subgraph.	Use WebGL renderer, server-side filtering, cluster summaries, progressive disclosure.
10,000+ nodes	Aggregated graph plus viewport queries and cached layouts.	Never send full graph to browser by default. Use scope, search, semantic zoom, server-side rollups.

- Compute layout only when the graph/view changes, not on every render.
- Cache layout by topology_id, scope hash, mode, overlay set, time window, and renderer.
- Store selected node/edge IDs separately from the full nodes/edges arrays to avoid unnecessary rerenders.
- Do not put graph algorithms inside React node components.
- For heavy overlays, precompute rollups server-side and stream only what the current view needs.
- Use WebGL overview renderer for exploration; open selected subgraph in React Flow for operator work.

14. Time, diff, golden path, and digital twin

Differentiator

Time-aware topology is how Correlix moves beyond pretty maps. Operators need to ask: what changed before the outage, what does live differ from intended, and how did the path look yesterday?

Required time fields

first_seen

last_seen

valid_from
 valid_to
 change_state: 'added' | 'removed' | 'changed' | 'unchanged'
 baseline_state
 golden_state
 current_state
 previous_state
 drift_summary
 confidence_trend

Capability	What the map should do
Historical diff	Show before/after topology for selected time window. Highlight added, removed, and changed nodes/edges.
Golden path	Compare live path to intended/golden path. Show match/diverge decision per hop.
Baseline path	Compare against previous known-good path or weekly/monthly baseline.
Change impact	Given a changed config/link/route, show affected paths, services, sites, and owners.
Replay	Replay topology and evidence around incident timeline.
Staleness detection	Mark links stale if last_seen exceeds source TTL; lower confidence over time.

15. Backend storage and API boundaries

Concern	Recommended component	Notes
Inventory and graph state	Postgres	Authoritative domain entities, resolved nodes/edges/groups, operator saved layouts, ownership, topology view metadata.
Flow/dependency analytics	ClickHouse	High-volume flow records, top talkers, dependency edges, sampled path evidence.
Metrics	VictoriaMetrics / Prometheus-compatible	Interface/device KPIs, utilization, error rates, health overlay inputs.
Logs/evidence	OpenSearch	Syslog, event evidence, RCA snippets, indexed raw references.
Cache and layout state	Redis	View caches, layout hashes, session-specific spotlight/filter state.
Graph projection service	Go service, optional worker	Build TopologyView contracts; run identity resolution, confidence scoring, and view scoping.

API boundary

The Graph API returns TopologyView. It does not return React Flow nodes. React Flow is only one renderer adapter.

16. Frontend implementation structure

src/features/topology/
 api/
 topologyApi.ts
 topologyTypes.ts
 graph/
 topologyFactTypes.ts
 topologyGraph.ts
 graphologyAdapter.ts
 graphAlgorithms.ts
 renderers/

react-flow/

TopologyCanvas.tsx

topologyToReactFlow.ts

nodes/DeviceNode.tsx SwitchNode.tsx RouterNode.tsx FirewallNode.tsx CloudNode.tsx GroupNode.tsx UnresolvedNode.tsx

edges/TopologyEdge.tsx PathHighlightEdge.tsx InferredEdge.tsx DegradedEdge.tsx BundledEdge.tsx

sigma/

SigmaTopologyView.tsx

topologyToSigma.ts

geo/

GeoTopologyMap.tsx

topologyToDeckLayers.ts

layout/

elkLayout.ts

layoutTypes.ts

layoutPresets.ts

savedLayoutStore.ts

workflows/

ExploreWorkflow.tsx

InvestigateWorkflow.tsx

PathTraceWorkflow.tsx

ChangeReviewWorkflow.tsx

CapacityWorkflow.tsx

DependencyWorkflow.tsx

ExecutiveGeoWorkflow.tsx

components/

TopologyToolbar.tsx TopologySearch.tsx TopologySideDrawer.tsx TopologyLegend.tsx TopologyMiniMap.tsx

OverlaySelector.tsx MapWorkflowSelector.tsx PathAnalysisPanel.tsx EvidencePanel.tsx ConfidencePanel.tsx

utils/

topologyMapper.ts topologyFilters.ts topologySearch.ts topologyOverlays.ts topologyHealth.ts semanticZoom.ts edgeBundling.ts

mock/

physicalTopology.ts pathTopology.ts incidentTopology.ts cloudTopology.ts enterpriseOverviewTopology.ts

Pattern	Rule
Mapper isolation	API graph -> renderer adapter -> renderer-specific nodes/edges/layers. Never mix API contract with React Flow internals.
Node components	Small, memoized, data-driven, reusable. No business logic or graph algorithms inside nodes.
Edges	Labels hidden by default; hover reveals ports, protocol, utilization, confidence, last seen, and evidence.
Selection	Node selection highlights neighbors and opens drawer. Edge selection opens evidence/confidence drawer.
Search	Search hostname, IP, interface, vendor, model, site, rack, role, owner, health, source fact, and evidence.
Overlays	Overlay renderer transforms graph attributes into visual state without mutating the domain graph.
Performance	Use density and zoom state to decide rendered detail level.

17. Claude Code skill pack v2

.claude/skills/topology-ui/

SKILL.md

design-tokens.md
 topology-context.md
 react-flow-patterns.md
 topology-contract.json
 renderer-strategy.md
 semantic-zoom.md
 topology-fact-model.md
 graph-scale-policy.md
 edge-bundling-rules.md
 time-diff-golden-path.md
 map-workflows.md
 performance-budget.md
 examples/
 physical-topology-sample.json
 incident-topology-sample.json
 cloud-topology-sample.json
 path-analysis-sample.json
 enterprise-overview-sample.json
 geo-wan-sample.json

File	What to put inside
SKILL.md	When to use the skill, product principles, required architecture, forbidden anti-patterns, quality bar.
renderer-strategy.md	React Flow vs Sigma/cosmos vs MapLibre/deck usage rules, adapter boundaries, scale thresholds.
semantic-zoom.md	Zoom levels, visible labels, collapsed objects, hover/click reveal rules.
topology-fact-model.md	Raw fact model, evidence model, confidence rules, source TTLs, identity resolution rules.
graph-scale-policy.md	Scale thresholds, progressive disclosure, server-side aggregation, cache policy.
edge-bundling-rules.md	Parallel link bundling, LAG rendering, circuit rendering, edge labels, bundle expansion.
time-diff-golden-path.md	Historical diff, live vs golden, baseline path, change impact, replay, stale evidence.
performance-budget.md	Node/edge limits, rerender rules, memoization, layout caching, WebGL handoff.

18. Implementation roadmap

Phase	Build	Outcome
Phase 1: Demo-ready physical map	React Flow canvas, ELK layered layout, custom device cards, curved/bundled edges, hover labels, side drawer, search, mini-map, mock LLDP/CDP/SNMP topology.	Premium physical topology demo with Datadog-like clarity and better evidence/confidence language.
Phase 2: NOC-grade map	Grouping/collapse, unresolved node handling, saved layouts, health/utilization/error overlays, evidence/confidence badges, edge bundling.	Operators can investigate with the map, not just look at it.
Phase 3: Path workflows	A-to-B path, hop list, ingress/egress interfaces, routing decisions, NAT/ACL/QoS/security checks, BGP-LS/LSDB support.	NetBrain-style troubleshooting path workspace.
Phase 4: Enterprise overview renderer	Sigma.js/cosmos.gl large graph mode, clustering, search, rollups, WebGL overview, open selected scope in React Flow.	Large enterprise scale without destroying React Flow performance.
Phase 5: Cloud/geo renderer	MapLibre/deck.gl site/region/cloud map, WAN circuits, provider paths, latency/capacity overlays.	Executive and WAN/cloud topology views.
Phase 6: Digital twin and change assurance	Golden state/config, live vs intended, historical replay, change impact map, baseline comparison.	Differentiated enterprise digital-twin topology capability.

19. Claude Code prompt

Build Correlix Topology Operating Canvas as a renderer-agnostic topology intelligence platform.

Do not build a generic graph visualization. Do not build a raw LLDP viewer. Build an extensible NOC operating canvas for physical topology, path analysis, incident RCA, dependency mapping, change review, and future digital twin workflows.

Public tools allowed:

- React Flow / xyflow for the scoped premium operator canvas
- ELK.js for deterministic layered/orthogonal/path layouts
- Graphology for client graph model and algorithms
- Cytoscape.js where graph analysis/headless operations are useful
- Sigma.js or cosmos.gl for large enterprise WebGL overview mode
- MapLibre GL JS + deck.gl for WAN/geo/cloud topology mode
- Tailwind + Radix/shadcn for enterprise controls
- Custom vendor-neutral SVG device icons

Architecture requirements:

1. Define TopologyFact and TopologyView contracts.
2. Create API graph -> renderer adapter mapping. Never expose React Flow internals from API.
3. Implement React Flow renderer for scoped workflows.
4. Keep placeholders/adapters for Sigma/cosmos and MapLibre/deck.
5. Implement ELK layout adapter and saved layout hooks.
6. Implement semantic zoom and edge bundling rules.
7. Implement evidence/confidence drawer for every edge.
8. Implement workflows: Explore, Investigate, Path Trace, Change Review, Capacity, Dependency, Executive/Geo.
9. Implement overlays: health, utilization, errors, routing changes, config drift, flow, RCA evidence, golden path delta, historical diff.
10. Every node and edge must support evidence, confidence, source facts, first_seen, last_seen, health, ownership, and change_state.

Quality bar:

- Premium enterprise NOC UI.
- Calm by default; intense only for selected, unhealthy, changed, or RCA-relevant objects.
- No hairball graph. Use grouping, zoom, search, scope, path focus, and expand-on-demand.
- Hide labels by default unless selected, searched, unhealthy, critical, or path-relevant.
- Keep unresolved devices visible but muted.
- Never draw links without evidence.
- Never use random force layout as default for physical/routed network topology.
- Do not animate everything.
- Do not hardcode positions except mock data fallback.
- Do not mix domain graph with renderer-specific state.

Deliver the file structure under src/features/topology/ exactly with renderer adapters, graph utilities, layout utilities, workflows, components, nodes, edges, overlays, mocks, and tests.

20. Acceptance criteria

Area	Acceptance criteria
Enterprise scale	A 5,000-node enterprise graph can be searched and explored in overview mode without loading every rich React node. A selected subgraph opens in React Flow.
Visual quality	Default map is calm, grouped, label-sparse, and readable. Selected path is visually strong. Unrelated graph dims.
Evidence	Every edge shows source, confidence, timestamp, and raw evidence reference. Missing evidence is explicit.
Path workflow	A-to-B path shows hop order, ingress/egress interfaces, routing decision, policy/NAT/security context, and golden/historical divergence.
Time/diff	Map can show added/removed/changed objects for a time window and compare live vs golden path.
Performance	React Flow remains scoped. Large overview uses WebGL renderer. Layout is cached and not recomputed every render.
Extensibility	New source types can be added as TopologyFacts without rewriting the renderer.
Operator action	Every selected object answers: what is it, why is it here, is it healthy, what changed, what depends on it, what evidence confirms it, and who owns the fix.

21. Public source references

- [1] Datadog Device Topology Map docs: https://docs.datadoghq.com/network_monitoring/devices/topology/
- [2] Datadog Cloud Network Monitoring Network Map docs: https://docs.datadoghq.com/network_monitoring/cloud_network_monitoring/network_map/
- [3] Dynatrace Smartscape topology view docs: <https://docs.dynatrace.com/docs/analyze-explore-automate/smartscape/smartscape-in-context-views/smartscape-view-topology>
- [4] Dynatrace Smartscape concepts docs: <https://docs.dynatrace.com/docs/analyze-explore-automate/smartscape/smartscape-concepts>
- [5] NetBrain Feature Handbook / dynamic maps and discovery: <https://www.netbrain.com/wp-content/uploads/2019/11/NetBrain-Feature-Handbook.pdf>
- [6] NetBrain Dynamic Map blog: <https://www.netbrain.com/blog/5-characteristics-define-dynamic-map/>
- [7] RFC 9552 - Distribution of Link-State and Traffic Engineering Information Using BGP: <https://datatracker.ietf.org/doc/rfc9552/>
- [8] OpenConfig gNMI specification: <https://www.openconfig.net/docs/gnmi/gnmi-specification/>
- [9] React Flow performance docs: <https://reactflow.dev/learn/advanced-use/performance>
- [10] React Flow examples / xyflow docs: <https://reactflow.dev/examples>
- [11] ELK.js GitHub project: <https://github.com/kieler/elkjs>
- [12] Eclipse ELK layered layout reference: <https://eclipse.dev/elk/reference/algorithms/org-eclipse-elk-layered.html>
- [13] Graphology docs: <https://graphology.github.io/>
- [14] Cytoscape.js docs: <https://js.cytoscape.org/>
- [15] Sigma.js docs: <https://www.sigmapjs.org/>
- [16] cosmos.gl project: <https://github.com/cosmosgl/graph>
- [17] MapLibre GL JS docs: <https://www.maplibre.org/maplibre-gl-js/docs/>
- [18] deck.gl docs: <https://deck.gl/>
- [19] MSAGL.js docs: <https://microsoft.github.io/msagljs/>
- [20] Nachmanson and Chen, Browsing Large Graphs with Tile Pyramids and Sleeve Routing in the Browser, 2026: <https://arxiv.org/abs/2605.17498>

Final product thesis

Correlix topology should be a topology intelligence platform. React Flow makes the scoped canvas beautiful. ELK makes it deterministic. Graphology/Cytoscape make it intelligent. Sigma/cosmos make it scalable. MapLibre/deck make WAN/cloud geography usable. The differentiated moat is the evidence-backed, time-aware graph behind all renderers.